Lecture 2

Architectural Styles

Part-I

Architecture Styles

Architectural Style

- This is the high-level structure of how the user interface, backend/ business logic, and data store are organized and interact.
- Think of it as the overall blueprint of the system

E.g. Layered architecture ,Client-server architecture ,Microservices architecture e.t.c

Architectural Pattern

- A reusable/low-level design solution to a common problem within a specific architectural style. like how to achieve scalability, performance, or security.

E.g. Model-View-Controller (MVC),Publish-Subscribe, Three-tier architecture

Design Pattern

- A reusable solution to a common design problem at the code level. It focuses on the interactions between objects and classes. E.g Factory ,Observe etc.

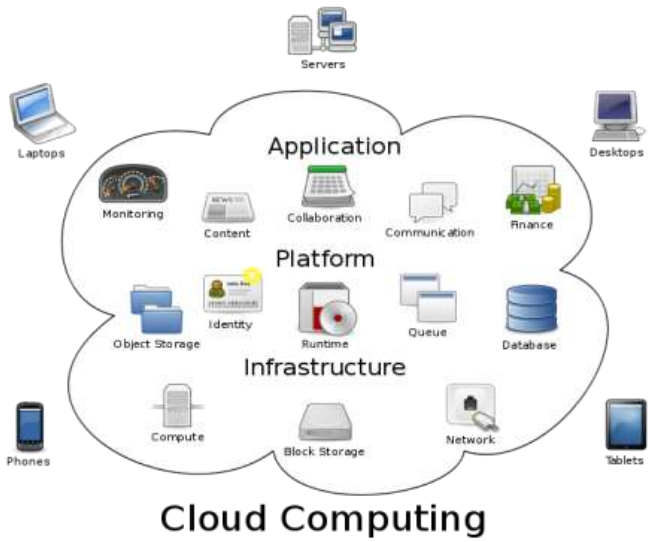
Architecture Styles

Scenario: Imagine you're designing a simple online store.

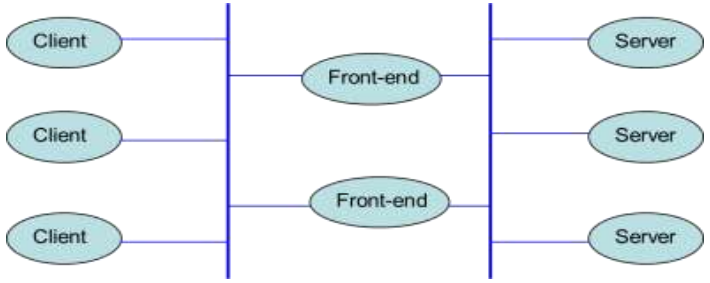
- Architectural Style: You decide to organize your store using ***Layered Architecture*** :
  - Presentation Layer: Handles user interface and interactions (e.g., website, mobile app).
  - Application Layer: Contains business logic (e.g., processing orders, managing inventory).
  - Data Layer: Stores and retrieves data (e.g., database).
- Architectural Pattern: you use the MVC pattern to separate concerns:
  - **Model:** Represents the data and business logic (e.g., product information, shopping cart).
  - **View:** Displays the data to the user (e.g., product page, checkout page).
  - **Controller:** Handles user input and updates the model and view (e.g., adding an item to the cart, processing an order).
- Design Pattern: You use a factory pattern to create different types of products (e.g., electronics, clothing) without having to write separate code for each type.

Architecture Styles

Various Patterns for Software Arch Layer Abstraction



Application System → System Pattern



Software Architecture → Architecture Pattern

Object/Class → Component Pattern (design pattern)

123-9475775 84775.99 …
01010101 0xFFFFEBAAA …

Data Structure → Data Model

Architecture Styles

Various Patterns for Software Arch Layer Abstraction

System Pattern: Highest level, encompassing the entire system, potentially including hardware, software, and even human elements, such as batch processing system, Web service architecture, SOA, cloud computing architecture, mobile application system, etc.

Architecture Pattern: Focuses on the software system's major components and their interactions. such as filter-pipe, C/S, B/S, MVC, broker, three-tier arch, etc.

Component Pattern (design pattern): The design and interaction of class/module within the software that carries certain type of structure and behavior

such as Factory, Observer, Singleton

Data Model: how data is structured, stored, and managed within the system.

such as Relational, NoSQL, Object-Oriented

Architecture Styles

Architectural views: A views or perspectives of the system that focus on particular aspects or concerns

4 + 1 view model of software architecture

- **Logical View *(structural view)*:** Focuses on the functional components and their relationships. such as classes, objects
- **Process View *(Behavioral view):*** Shows how the system behaves at runtime, including processes and threads. Such as activity diagrams and sequence diagrams
- **Development View *(implementation view)*:** Describes the organization of the code, including modules and packages.
- **Physical View *(Deployment view)*:** Shows how the system is deployed on hardware and infrastructure. Such as Deployment diagrams. Such as Component diagrams and package diagrams

•Use cases or scenarios (+1):- help to tie the four views together and demonstrate how they work in practice

Architecture Styles

Example: E-commerce System

Microservices Architecture:

- **Logical View:** Many small, independent services (catalog service, cart service, Order service etc.).
- **Process View:** Each microservice runs as its own independent process, communicating over a network.
- **Physical View:** Each microservice can be deployed independently on different servers or containers.
- **Development View:** Separate codebases for each microservice. Small, independent teams can work on different services.

Architecture Styles

Types of Software Architecture

- Data Flow Architecture
  - batch processing
  - filter/pipe
- Data Centered Architecture
  - repository design
  - blackboard architecture
- Hierarchical Architecture
  - layered architecture
  - main/subroutine
- Interaction Oriented Architecture
  - MVC (Model-View-Control)
  - PAC (Presentation-Abstraction-Control)
- Object Oriented Architecture
  - OOAD
  - UML model
- Component- Based Architecture
  - framework approach
- Distributed Architecture
  - client/server,
  - peer-to-peer
  - multi-tier
- Service-Oriented Architecture
  - integration with business process

Architecture Styles

Architectural Styles

1. Client/Server Architecture Style
2. Layered Architecture Style
3. Pipeline Architecture Style
4. Microkernel Architecture Style
5. Event-Driven Architecture Style
6. Space-Based Architecture Style
7. Orchestration-Driven Service-Oriented Architecture
8. Micro services Architecture
9. Architecture in the Edge/ cloud
10. Big data Architecture
11. Machine Learning Architecture

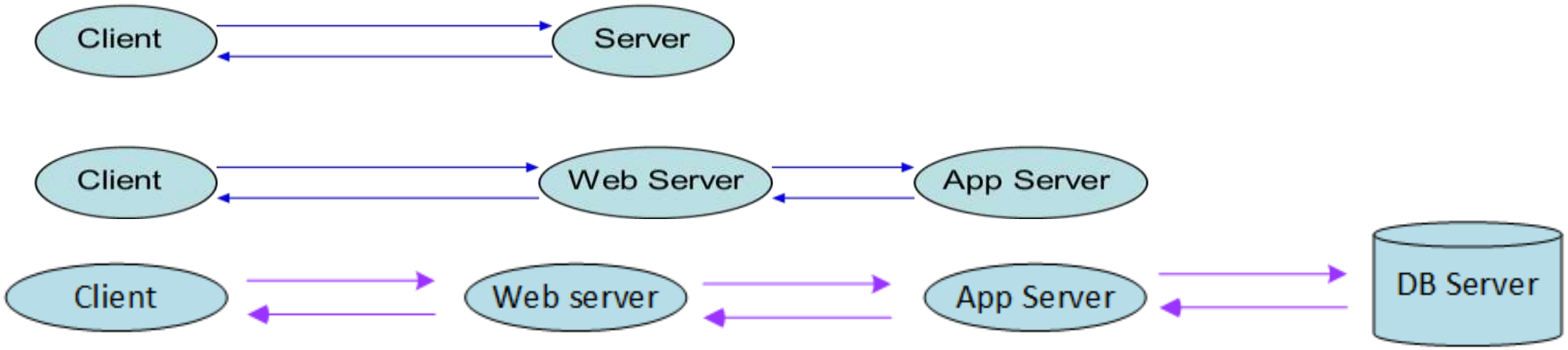
Architecture Styles

1. Client Server Architecture

Consists of two main components:

- **Client:** Requests services or resources. Examples: Web browsers, email clients, mobile apps.
- **Server:** Provides services or resources. Examples: Web servers,, App servers, database servers

- Clients make remote procedure calls to servers using a protocol like http, then wait for reply.
- The server often handles both application logic (business rules, data processing) and data storage.



Architecture Styles

Pros of C/S Architecture

- Client program and Server application are running on separate machines, which is good to data distribution
- Architecture transparency – Client doesn't need to know where Server is located
- Client program and Server application may run on distinct platforms, with the focus on various tasks
- Client program is independent to and geographically separate from Server application, which makes the deployment of system is considerably flexible
- Suitable for Small-Scale Applications and simple to understand and implement(Simplicity)
- Faster response times for small number of client request due Direct Communication

Architecture Styles

Cons of C/S Architecture

- **Single Point of Failure:** If the server goes down, the entire system is affected.
- **Tight Coupling:** Changes to the server can impact clients, and vice-versa.
- **Security Risks:** Sensitive logic and data are on the same machine, increasing vulnerability.
- **Limited Scalability:** Scaling can be challenging due to the network bandwidth and the server handles multiple roles.
- **Heterogeneous** standard, platform and technology used for the Client **program are** tends to high **development and maintenance cost**

Solution: **3-Tier Architecture:** Introduces a middle tier, resulting in three logical layers:

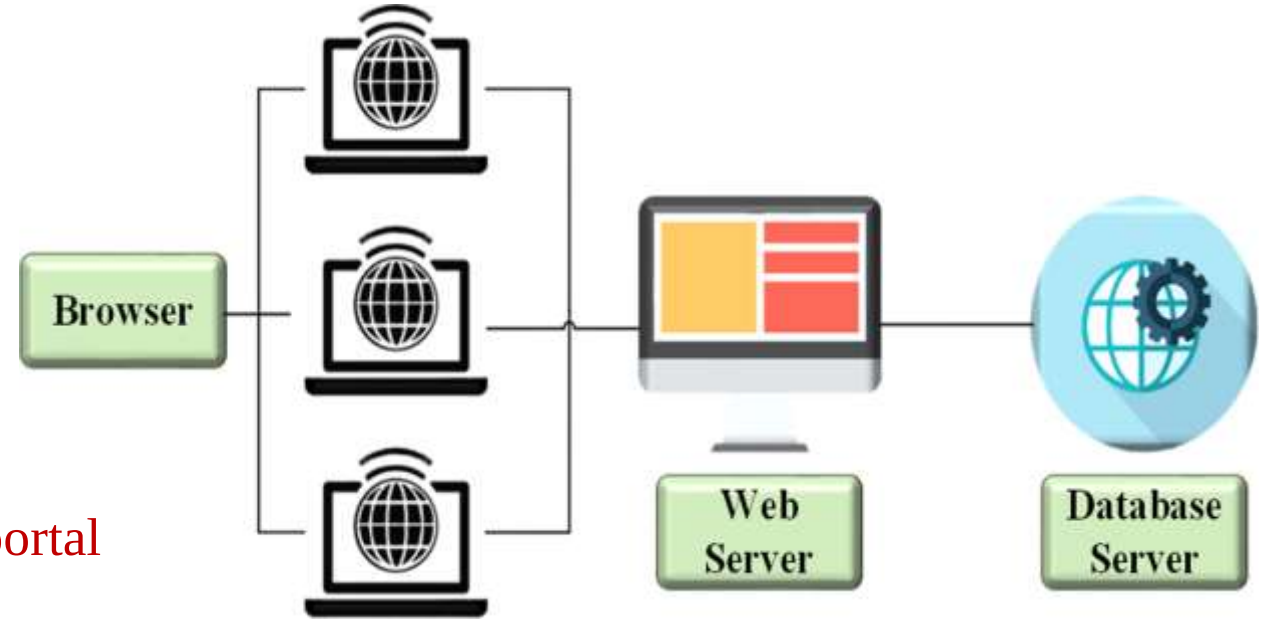
- ***Presentation Tier (Tier 1):*** *The client (browser, app) that the user interacts with.*
- ***Application Tier (Tier 2):*** *Handles application logic, data processing, and interacts with the data tier.*
- ***Data Tier (Tier 3):*** *Stores and manages the data (database server).*

Browser-Server (B/S) Architecture

A specific type of Client-Server architecture where the **client is a web browser**.

Examples:

- Browsing a website like Google or Amazon.
- Using a web-based email service like Gmail or Outlook.com.
- Interacting with a web application like a banking portal or social media platform.



Architecture Styles

Pros of B/S Architecture

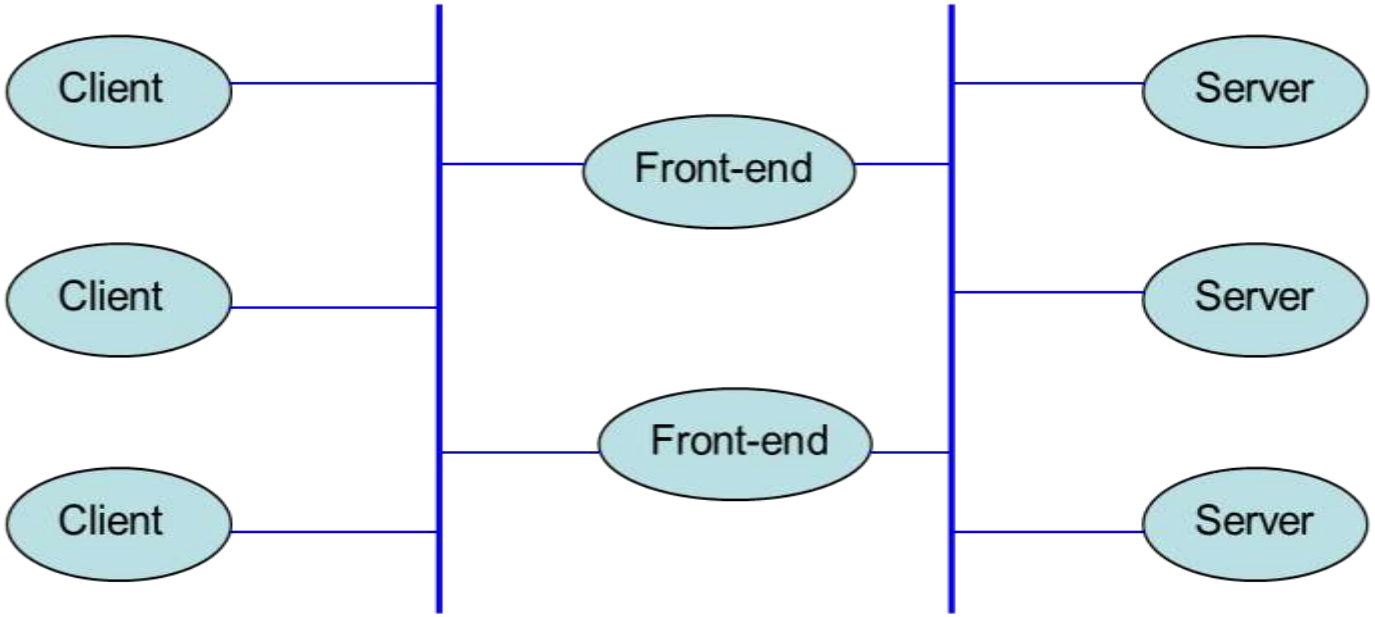
- The Client is a simple browser program, which is easy to use and maintain
- Based on the standard HTTP protocol， it is easy to build a cross-platform s/w system. The Server is totally separated from the Client
- Development focus is on the Server application, i.e. at the backend, which greatly reduces the development and maintenance cost
- While basic B/S has scaling limitations, with the addition of techniques like load balancing, caching, and database replication, B/S architectures can be made highly scalable to handle a large number of users

Cons of B/S Architecture

- **Scalability Bottlenecks:** The server can become overwhelmed with requests as the number of clients increases.
- **Single Point of Failure:** If the server fails, the entire application becomes unavailable.
- **Network Dependency:** Performance is heavily reliant on the network connection between clients and the server.
- **Security Vulnerabilities:** The server can be a prime target for attacks, and data transmitted over the network can be vulnerable.
- **Maintenance Challenges:** Server-side code can become complex and difficult to maintain as the application grows.

An Improvement to B/S Architecture

Multi-tier architecture

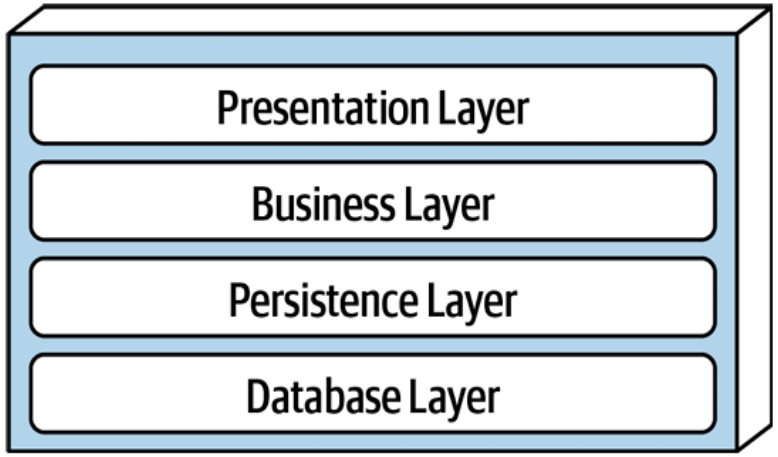


Architecture Styles

2. layered architecture – Structural Model

The application is divided into horizontal layers, each responsible for a specific function.

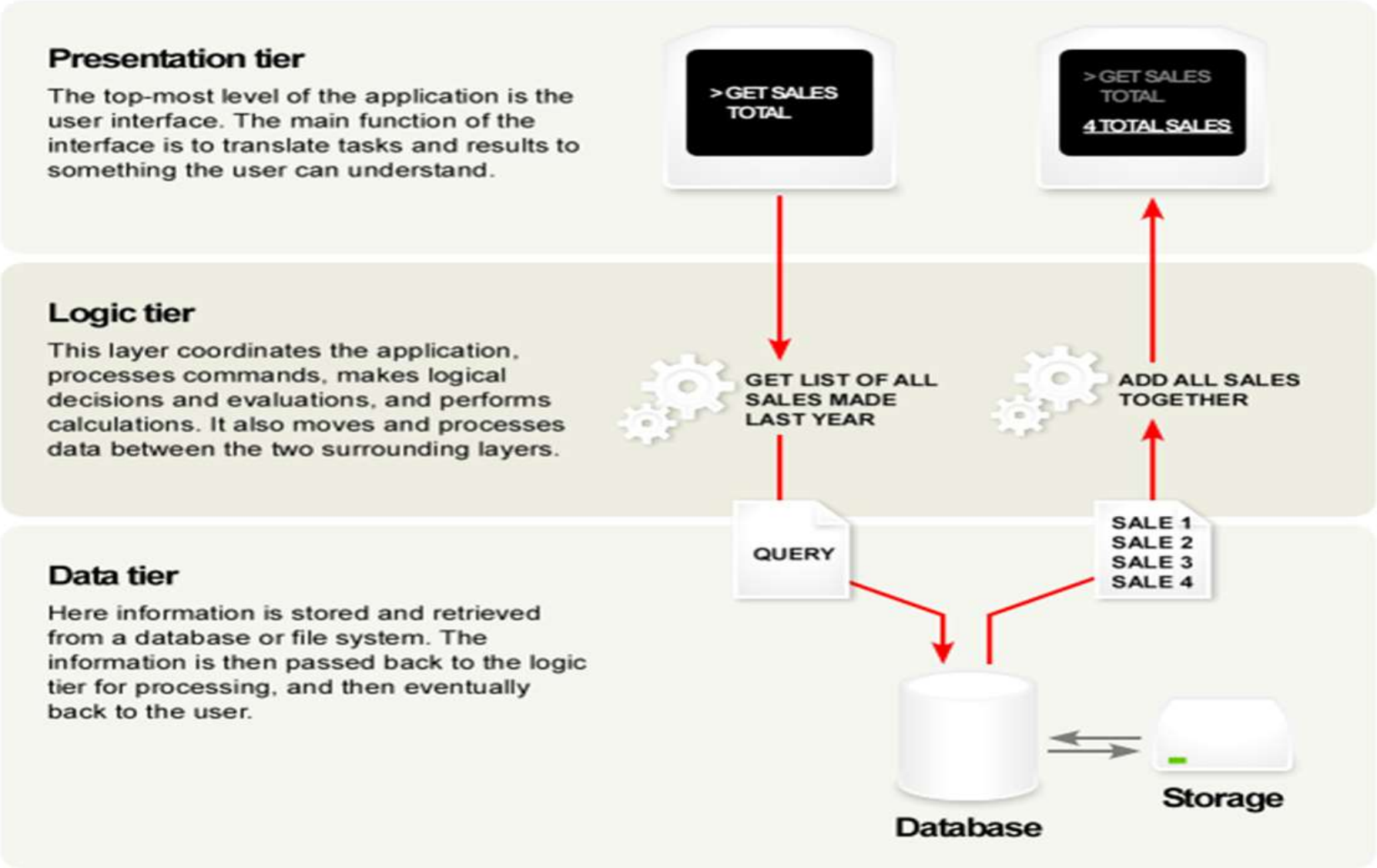
Common layers include:



- **Presentation Layer (UI):** Handles user interaction, displays information, and communicates with the user (e.g., a website's user interface).
- **Business Layer (Logic):** Implements the core business rules and logic of the application (e.g., calculations, validations, workflows).
- **Persistence Layer (Data Access):** Handles the storage and retrieval of data (e.g., interacting with a database).
- **Database Layer:** The actual database system where the data is stored.

Architecture Styles

layered architecture



Architecture Styles

2. layered architecture …

Pros of Layered Architecture:

- **Simplicity :** Easy to understand and implement, making it a common choice for many applications.
- **Low Cost (Initially):** Simpler development and maintenance, especially for smaller applications.
- **Natural Mapping to Organizational Structure:** often aligns with typical development teams (UI, backend, database), facilitating communication and project management.
- **Maintainability:**Changes to one layer do not affect layers above
- **Reusability:** Components within a layer *can* be reused in other parts of the application.(with same interface).

Architecture Styles

2. layered architecture …

Cons of Layered Architecture:

- **Scalability Challenges:** Scaling a layered application can be difficult. You often have to scale the entire application, even if only one part needs more resources.
- **Performance Bottlenecks:** Requests have to travel through multiple layers, which can add overhead and slow things down.
- **Lack of Agility:** Making changes that affect multiple layers can be time-consuming and complex, making it hard to respond quickly to changing needs.,
- **Limited Fault Tolerance:** A failure in one part of the monolith can bring down the entire application.
- **Poor Testability:** Changes in one part of the monolith can require retesting the entire application.

Architecture Styles

2. layered architecture …

When to Use Layered Architecture:

- Small, simple applications or websites.
- Situations with tight budget and time constraints.
- When the architecture is still being analyzed.

When to Avoid Layered Architecture:

- Large, complex applications.
- Applications requiring high scalability, performance, and fault tolerance.
- When agility and frequent deployments are critical.

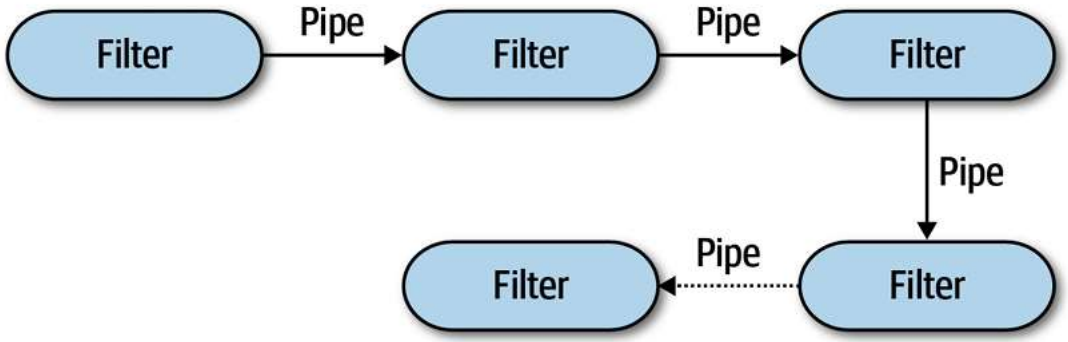
Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★

3. Pipe-Filter (pipeline architecture)

- Every functional unit has a group of input/output interface.
  - Filters :Functional building blocks
  - Pipes :Output/input connections, Unidirectional communication channels between filters
- Filter is also a stateless running unit that does not have impact to other filter's state
  - This makes filters independent to that of other filters', significantly improving modularity.

Topology



Architecture Styles

3. Pipe-Filter…

Pros of pipeline Architecture:

- **Simplicity and Modularity:** The clear separation of concerns into individual filters makes the architecture easy to understand, implement, and maintain. Filters can be developed, tested, and deployed independently.
- **Reusability:** As long as they support the same pipe interface, filters can be reused in different pipelines for various tasks, promoting code efficiency and reducing development time.
- **Maintainability:** Changes to one filter generally don't affect other filters
- **Flexibility:** New filters can be easily added to the pipeline to extend functionality or modify existing behavior.
- **Cost-Effective:** Compared to more complex distributed architectures, less expensive to build and operate.

Architecture Styles

3. Pipe-Filter…

Cons of pipeline Architecture :

- **Limited Scalability and Elasticity:** The monolithic nature of typical pipeline deployments makes it challenging to scale individual parts of the system independently.
- **Poor Fault Tolerance:** A failure in one filter can bring down the entire pipeline due to the monolithic nature. Recovery times can be long.
- **Limited Testability and Deployability:** While better than purely layered architectures, testing and deploying the entire monolith can still be cumbersome and time-consuming.
- **Not Suitable for Complex Interactions:** Best suited for simple, linear data processing flows.
  - *The pipeline architecture is generally **not** the most appropriate choice for online processing systems that involve significant user interactions, especially those requiring low latency and high availability*

Architecture Styles

3. Pipe-Filter…

When used

- **Batch Processing:** Processing a large set of data in a sequential manner.
- **Data Transformation:** Converting data from one format to another (e.g., CSV to XML).
- **Compilers:** The stages of compilation (lexing, parsing, semantic analysis, code generation) can be viewed as a pipeline.

Not Good For:

- Complex Interactions between components
- Distributed systems requiring high availability, scalability, and fault tolerance.
- Real-time processing of massive data streams might also be challenging.

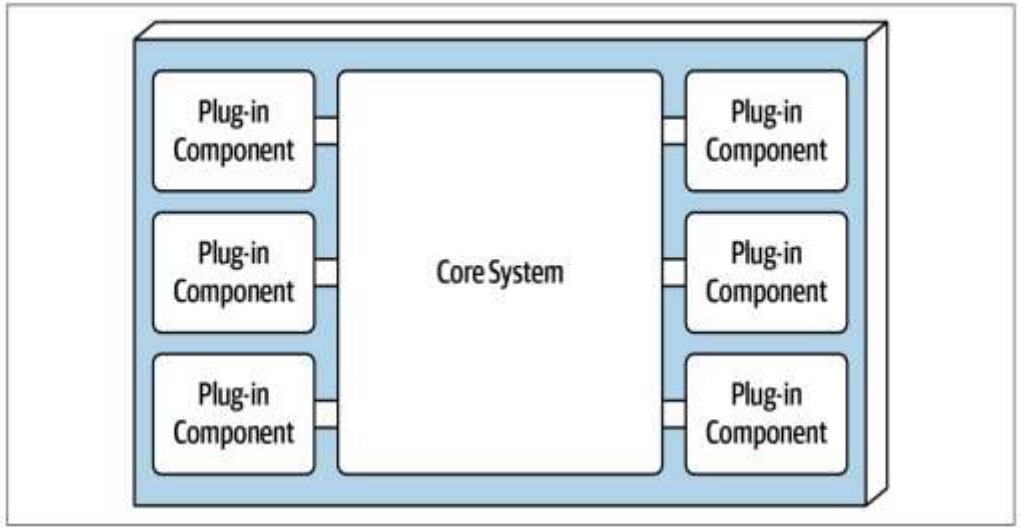
Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★★

4. Microkernel Architecture Style

Also referred to as the *plug-in architecture*

- Relatively simple monolithic architecture consisting of two architecture components:
  - A core system and
  - Plug-in components
- Application logic is divided between independent plug-in components and the basic core system.
- Providing extensibility, adaptability, and isolation of application features and custom processing logic.



Basic components of the microkernel architecture style

4. Microkernel Architecture Style…

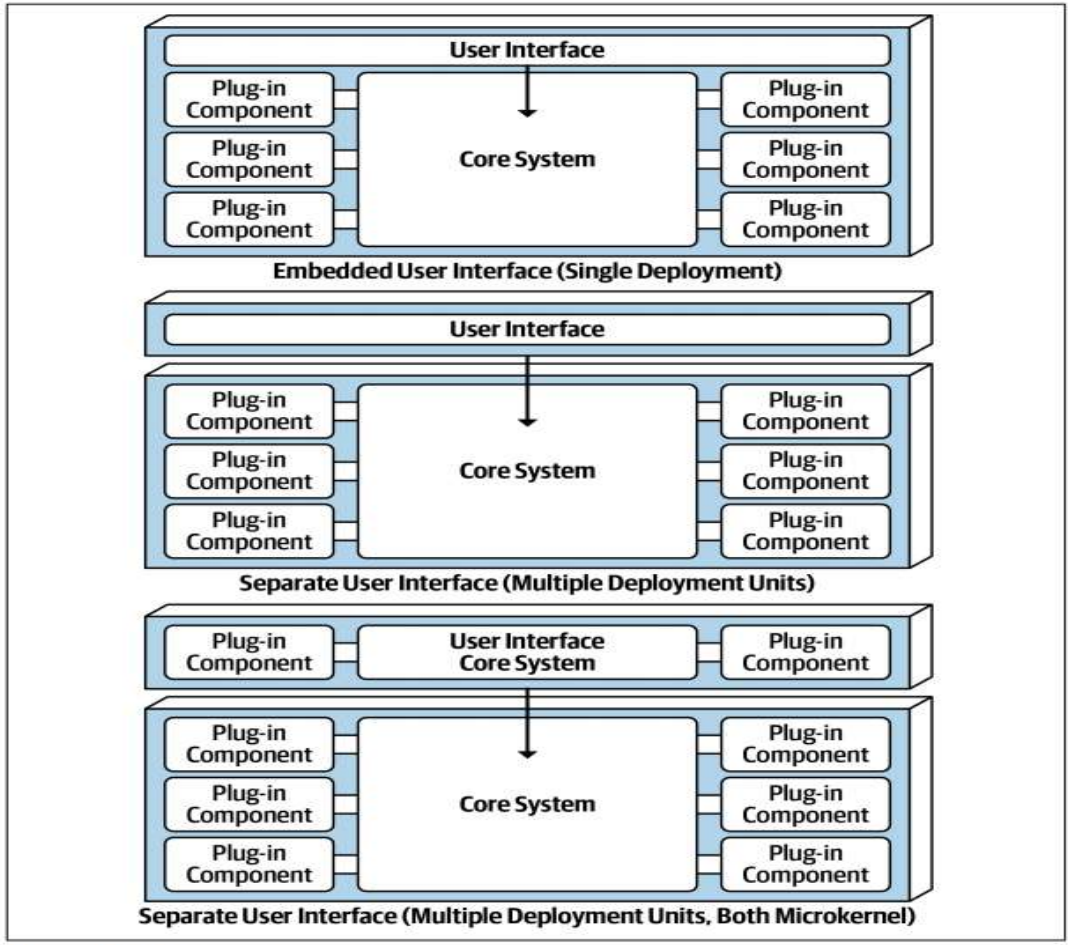
Core System

- The core system is formally defined as the minimal functionality required to run the plug-in components.
- Depending on the size and complexity, the core system can be implemented as a layered architecture or a modular monolith
- Example **Payment Processing** is a **core system**, payment methods such as Pay pal, credit card, store gift are domain specific component plug-ins in **Payment Processing Core system.**

4. Microkernel Architecture Style…

Core System…

- The presentation layer/ User interface of the core system:
  - ❑ Embedded within the core system
  - ❑ Implemented as a separate user interface, with the core system providing backend services
  - ❑ Implemented as a microkernel architecture style.



4. Microkernel Architecture Style…

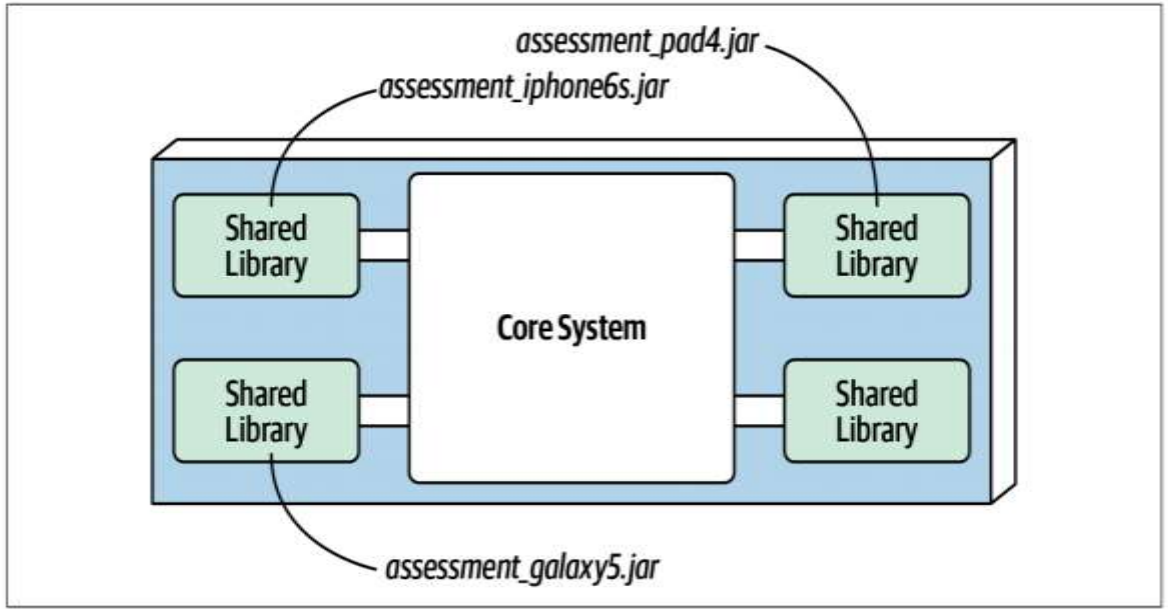
Plug-In Components

- Standalone, independent components that contains specialized processing.
- Should be independent of each other and have no dependencies between them.
- The communication with the core system is generally point-to-point, REST, Messaging.
- Can be either compile-based or runtime-based.
- **Runtime plug-in components** can be added or removed at runtime without having to redeploy the core system or other plug-ins.
    - Plug-ins are managed through frameworks such as Open Service Gateway Initiative (OSGi) for Java, Prism (.NET).
- **Compile-based plug-in components** are much simpler to manage but require the entire monolithic application to be redeployed when modified, added, or removed

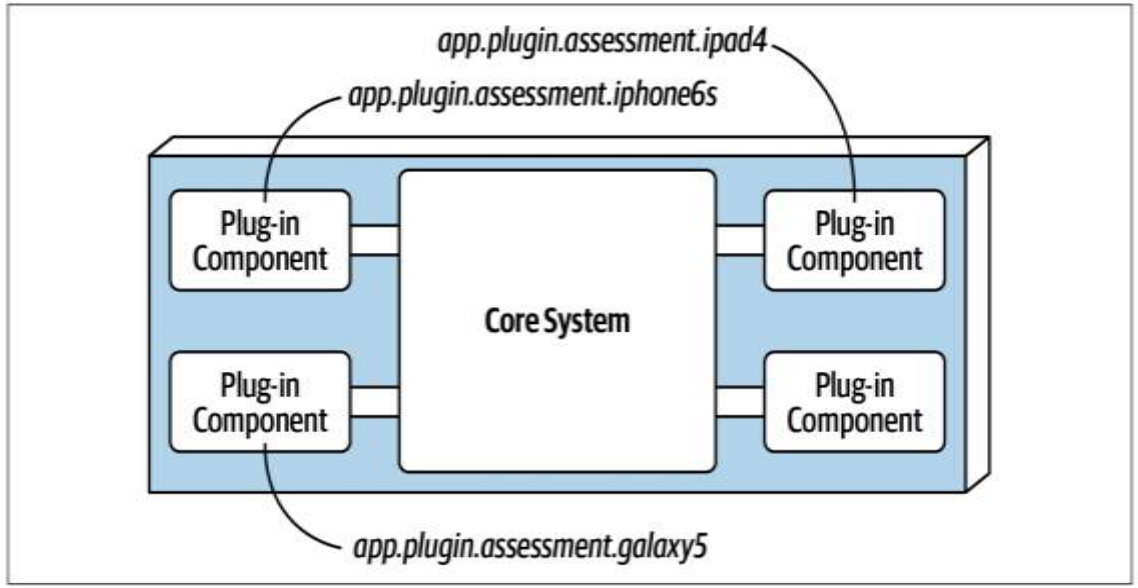
4. Microkernel Architecture Style…

Plug-In Components…

- Point-to-point plug-in components can be implemented as shared libraries (such as a JAR, DLL, or Gem), package names in Java, or namespaces in C#.



Shared library *plug-in implementation*

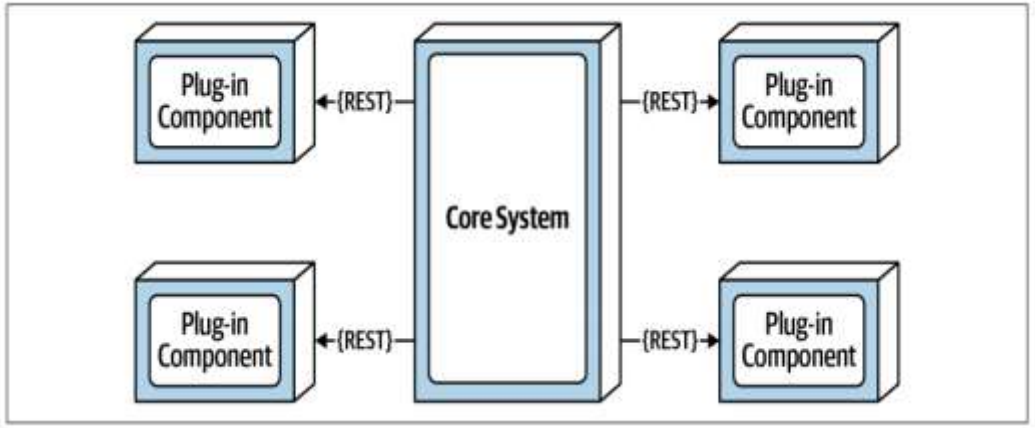


Package or namespace *plug-in implementation*

4. Microkernel Architecture Style…

Plug-In Components…

- Plug-in components can also be a REST or messaging as a means to invoke plug-in functionality, with each plug-in being a standalone service
- Every request must first go through the core system to get to the plug-in service



Remote plug-in access using REST

- The benefits of remote access approach to accessing plug-in components provide better overall component decoupling.
- Allows for better scalability and throughput, and allows runtime changes without any special frameworks like OSGi.
- Remote plug-in access turns the microkernel architecture into a distributed architecture rather than a monolithic. Which add complexity on implementation and deployment.

Plug-In Components…

Registry

- A pug-ins registry mainly assists the core system to know/locate about which plug-in modules are available and how to get to them.
- Registry contains information about each plug-in module, including things like its name, data contract (input data and output data), and remote access protocol details.

Contracts

- The contracts between the plug-in components and the core system are usually standard across a domain of plug-in components.
- Contracts include the behavior, input data, and output data returned from the plug-in component.
- Plug-in contracts can be implemented in XML, JSON, or even objects passed back and forth between the plug-in and the core system.

4. Microkernel Architecture Style…

Pros:

- **Extensibility:** Adding new features is simplified through plug-ins, without altering the core system.
- **Flexibility:** Plug-ins can be modified or replaced independently.
- **Maintainability:** Easier to manage and update.
- **Fault Isolation:** Failures in plug-ins are less likely to bring down the entire system, enhancing reliability.
- **Testability:** Plug-ins can be tested independently, streamlining the development process.
- **Overall cost:** considered relatively low, likely due to the simplicity of the core system and the ability to reuse plugins.

4. Microkernel Architecture Style…

Cons:

- **Performance Overhead:** Communication between the core system and plug-ins can introduce performance bottlenecks.
- **Complexity:** Managing a large number of plug-ins and their dependencies can become complex
- **Deployment Challenges:** Deploying and managing numerous plug-ins can be complex, especially in distributed environments.
- **Elasticity:** With a single quantum (deployable unit), the system cannot easily scale individual components.
- **Scalability:** As with elasticity, scalability is a major weakness. The single quantum limits the ability to scale individual components.

4. Microkernel Architecture Style…

When to Use:

- **Applications with Evolving Requirements:** When frequent changes and additions are expected.
- **Product-Based Applications:** Where customization and extensibility are crucial (e.g., Eclipse IDE, web browsers such as Chrome and Firefox).
- **Complex Business Applications:** That require handling diverse rules and variations (e.g., insurance claims, tax software).

Key Considerations:

- A well-defined registry is crucial for managing plug-ins effectively.
- Clear and consistent contracts between the core system and plug-ins are essential for interoperability.

4. Microkernel Architecture Style…

Architecture Characteristics Ratings

One star **(*) lowest support** and five start **(*****) strongest support** rating

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★★
Reliability	★★★
Scalability	★
Simplicity	★★★★
Testability	★★★

Thanks ! . .